

**An Agentic Artificial Intelligence Framework for Automated Disruption Recovery and
Passenger Re-accommodation for Kenya Airways**

[Capstone Project]

Otieno Bill Ritchie

163362

ICS 4C

Supervisor Name

Kelvin Ouma

**An Informatics and Computer Science Project Proposal Submitted to the School of
Computing and Engineering Sciences in partial fulfillment of the requirements for the
award of a Degree in Bachelor of Science in Informatics and Computer Science**

School of Computing and Engineering Sciences

Strathmore University

Nairobi, Kenya

June 2026

Declaration and Approval

I declare that this project proposal has not been submitted to Strathmore University or any other University for the award of a Degree in Bachelor of Science in Informatics and Computer Science or any other Degree. To the best of my knowledge and belief, the research proposal contains no material previously published or written by another person except where due reference is made in the research proposal itself.

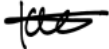
Student Name: Otieno, Bill Ritchie

Admission Number: 163362

Sign: 

Date: 12th May,2026.

Supervisor Name: Kelvin Ouma

Sign: 

Date: 12th May,2026.

The Proposal of **Otieno, Bill Ritchie** has been reviewed and approved by **Mr. Kelvin Ouma**

Supervisor Signature:  Date: 12th May,2026

Acknowledgement

I would like to extend my heartfelt gratitude to my project supervisor for providing invaluable academic guidance, technical critiques, and structural direction throughout the formulation of this research proposal. Special appreciation is also extended to the faculty members of the School of Computing and Engineering Sciences at Strathmore University for cultivating a rigorous research environment. Finally, I acknowledge my peers and family for their steadfast encouragement during the development of this architectural framework.

Abstract

As Kenya Airways (KQ) navigates a critical operational recovery phase, the real-time management of flight disruptions, officially known as Irregular Operations (IROPS), remains a primary threat to profitability and customer retention. Current manual intervention strategies for passenger re-accommodation are characterized by significant time lags, high labor costs, and transactional errors, leading to extreme passenger dissatisfaction and high voucher expenditures. This project develops an autonomous, web-based, and telephony-integrated AI framework utilizing Retrieval-Augmented Generation (RAG) and Agentic Workflows to manage the end-to-end passenger disruption lifecycle.

By integrating a unified Supabase (PostgreSQL) architecture, the system provides hallucination-free, policy-compliant guidance grounded strictly in official Kenya Airways Conditions of Carriage and international civil aviation regulations. The system continuously polls real-time flight status via external global distribution system APIs to proactively trigger recovery workflows, including the automated generation of digital vouchers and alternative flight re-booking. A core computer science innovation within this framework is the implementation of ACID-compliant atomic transactions with row-level database locking to guarantee inventory integrity, effectively preventing double-booking anomalies during peak, high-concurrency disruption events. Furthermore, the architecture features an automated semantic distance thresholding algorithm that detects out-of-scope inquiries and generates a concise diagnostic case brief to execute a seamless, context-aware hand-off to human customer care agents via a telephony voice network. This system bridges the gap between high-stress operational crises and data-driven customer service automation.

Keywords: *Agentic AI, Disruption Recovery, Retrieval-Augmented Generation (RAG), Passenger Re-accommodation, ACID-compliant Transactions, Irregular Operations (IROPS), Multimodal Architecture.*

Table of Content

Chapter 1: Introduction.....	1
1.1 Background of the Study	1
1.2 Problem Statement	2
1.3 General Objective	3
1.3.1 Specific Objectives	3
1.3.2 Research Questions	4
1.4 Justifications of the Study.....	4
1.5 Scope and Limitations.....	5
1.5.1 Scope of the Project	5
1.5.2 Limitations in the Project.....	5
1.5.3 Delimitations in the Project	6
Chapter 2: Literature Review.....	7
2.1 Introduction.....	7
2.2 Current State of Disruption Recovery in Aviation	7
2.3 Related Works in Airline Disruption Management	7
2.4 Traditional Telephone Menus (Legacy IVR Systems)	7
2.3.2 Basic Rule-Based Chatbots.....	8
2.3.3 Standard Flight Booking Databases (Legacy GDS)	9
2.4 Gaps in Related Works	10
2.5 Conceptual Framework.....	11
Chapter 3: Development Methodology	13
3.1 Introduction.....	13
3.2 Research Paradigm.....	13
3.2.1 Data Acquisition	13

3.2.2 Data Preprocessing.....	13
3.2.3 Model Training	14
3.2.4 Model Validation and Testing	14
3.3 Software Development Methodology	15
3.3.1 Requirements Elicitation.....	15
3.3.2 Sprint Planning.....	16
3.3.4 Daily Standup and Tracking	16
3.3.5 Sprint Review.....	16
3.3.6 Sprint Retrospective.....	16
3.3.7 Coding and Development	16
3.3.8 Testing and Deployment.....	17
3.4 System Analysis and Design.....	17
3.4.1 Use Case Diagram.....	17
3.4.2 Class Diagram.....	17
3.4.3 Entity Relationship Diagram (ERD).....	18
3.4.4 Database Schema	18
3.4.5 System Architecture.....	18
3.4.6 Wireframes.....	18
3.5 System Development Tools and Techniques	19
3.5.1Flutter	19
3.5.2 FastAPI	19
3.5.3 Supabase	19
3.5.4 Technique 1: Data-retrieval check	19
3.5.5 Technique 2: Database lock during transactions	19
3.5.6 Technique 3: Assistant Networks	20

3.6 System Deliverables.....	20
3.6.1 System Documentation	20
3.6.2 Authentication Module	20
3.6.3 Text and speech ingestion.....	20
3.6.4 Factual policy Engine	20
3.6.5 Secure transaction	21
Reference	22
Appendix.....	24
Appendix 1 : Gantt Chart.....	24
Appendix 2 : Similarity Report.....	2

List of figures

Fig. 2.1: Conceptual framework	12
Fig. 3.1: Agile Methodology.....	15

List of tables

Table 2.1: Summary of Existing Airline Systems and Their Flaws	10
--	----

List of Abbreviations

ACID – Atomicity, Consistency, Isolation, Durability

AI – Artificial Intelligence

API – Application Programming Interface

APA – American Psychological Association

BERT – Bidirectional Encoder Representations from Transformers

CRUD – Create, Read, Update, Delete

CS – Computer Science

DBMS – Database Management System

ERD – Entity Relationship Diagram

GDS – Global Distribution System

IATA – International Air Transport Association

IROPS – Irregular Operations

IVR – Interactive Voice Response

JSON – JavaScript Object Notation

KQ – Kenya Airways

LLM – Large Language Model

NLP – Natural Language Processing

QR – Quick Response (e.g., QR Codes)

RAG – Retrieval-Augmented Generation

REST – Representational State Transfer

S-BERT – Sentence Bidirectional Encoder Representations from Transformers

STT – Speech-to-Text

TTS – Text-to-Speech

UI – User Interface

UML – Unified Modeling Language

URI – Uniform Resource Identifier

Chapter 1: Introduction

1.1 Background of the Study

The commercial aviation sector serves as a backbone for global trade, economic integration, and tourism. Within the African aviation ecosystem, Kenya Airways (KQ) represents a vital hub network operating out of Jomo Kenyatta International Airport (JKIA) in Nairobi. As airlines scale passenger capacities to meet post-recovery demands, they face persistent challenges related to Irregular Operations (IROPS)—defined as any flight deviation from the published schedule caused by severe weather patterns, air traffic control restrictions, unexpected engineering technicalities, or global supply chain backlogs (Cook & Lawes, 2012). Managing these disruptions efficiently is a highly complex task that directly impacts an airline's financial bottom line and long-term brand equity (BHI Research Group, 2025)

A delayed or cancelled flight means that hundreds of passengers need to be processed instantly at several service touchpoints: itinerary changes, visa and health documentation verification, meal adjustments and hotel accommodation tracking. Historically, these recovery procedures have been heavily based upon legacy Global Distribution Systems (GDS) manipulated manually by ground station agents and telephone call center personnel. The methodology in operations causes significant processing latency which leads to crowded airport terminals, long customer queue times and increased operational friction . Early computational solutions to this problem tried to do so with deterministic integer linear programming models, but these models assume static environments and are not suitable for the dynamic real-time communication needs of stranded passengers .

The fast development of Artificial Intelligence (AI), particularly large language models (LLMs), has created a new field of complex computational systems called Agentic Workflows. Different from classic static chatbots that are hard-coded into a rigid decision tree, an Agentic AI system can internally reason, assess complex situations, break down a multi-step goal into separate executable tasks, and invoke external application programming interfaces (APIs) to independently change data systems. When coupled with Retrieval-Augmented Generation (RAG) that injects real-time, verified contextual knowledge into the prompt context window, an agent can safely traverse corporate policy boundaries without fear of generating inaccurate outputs, popularly known as AI hallucinations.

Advancements in conversational AI have been paralleled by the maturity of modern cloud database architectures to support strong concurrency controls. Systems like Supabase are built on top of relational PostgreSQL database structures. They are ACID compliant and come with advanced data extensions that provide native support for atomicity, consistency, isolation and durability. The convergence of these technologies makes it mathematically and logically possible to deploy autonomous systems capable of high-stakes transaction processing, such as seat allocation, where resource availability must be protected with absolute precision across multiple channels at the same time.

1.2 Problem Statement

As conversational AI has evolved, modern cloud database architectures have matured to support strong concurrency controls. Despite large digital investments, the Supabase and similar systems are built on top of the relational database structure of PostgreSQL. They are ACID compliant and have advanced data extensions that offer native support for atomicity, consistency, isolation and durability. The convergence of these technologies means that it is mathematically and logically possible to implement autonomous systems that can handle high-stakes transaction processing, like seat allocation, where the availability of resources needs to be protected with absolute precision across multiple channels at the same time. Management of flight disruptions at Kenya Airways still suffers from extreme inefficiencies. Manual customer care pipelines cannot scale fast enough to meet passenger demands during major unexpected delays or structural flight cancellations at central hubs like JKIA (Cook & Lawes, 2012). This creates a huge “speed-to-lead” performance gap, where stranded passengers are left without communication for hours before confirmation of alternative travel arrangements. This processing lag has serious financial implications for the carrier including statutory compensation claims, high overnight hotel voucher liabilities, and a drop in consumer trust as seen in lower Net Promoter Scores (NPS) (BHI Research Group, 2025).

Furthermore, the present automated communication channels used by the airline industry do not solve this crisis. Traditional Interactive Voice Response (IVR) telephone support systems provide inflexible, multi-tiered phone menus that do not interpret complex customer conditions. Simple conversational chatbots are not programmatically integrated with the underlying ticketing databases. These standalone chatbots are highly susceptible to semantic

hallucinations because they lack rigorous domain-specific guardrails. In an aviation setting, this can lead to unlawful promises of premium ticket upgrades, wrong visa validation or illegal compensation compliance with grave legal and regulatory liabilities if a chatbot hallucinates and is not grounded.

The technical problem of transaction concurrency only adds to this problem. In the case of a sudden cancellation of a flight involving a high-capacity aircraft, hundreds of stranded passengers may at the same time open their mobile devices to compete for a highly limited supply of available seats on alternate flights. Standard database paradigms lacking strict, serialised isolation levels suffer from operational race conditions . This results in severe data collisions where the exact same physical seat is double-booked across different user interfaces at the exact same millisecond (Silberschatz et al., 2020). Thus, there is still a critical research and engineering gap for a unified, multimodal framework that can automate policy-compliant customer recovery while performing collision-free database transactions.

1.3 General Objective

To design, develop, and validate an Agentic AI Framework featuring integrated web and telephony multimodal interfaces to automate disruption recovery and secure collision-free passenger re-accommodation at Kenya Airways.

1.3.1 Specific Objectives

- i. To identify and analyze the structural challenges and operational bottlenecks experienced by passengers and airlines during Irregular Operations (IROPS).
- ii. To design an information retrieval backend utilizing Retrieval-Augmented Generation (RAG) to ensure hallucination-free passenger guidance.
- iii. Develop an autonomous application architecture that supports ACID-compliant atomic transactions and row-level database locking for zero-collision seat inventory allocation.
- iv. To mathematically evaluate an intelligent semantic routing and distance thresholding algorithm to detect out-of-scope customer inquiries, actively measuring false-positive rejection rates against a mathematically optimized F_β score to execute seamless hand-offs to human support agents.

1.3.2 Research Questions

- i. What are the key operational bottlenecks and systemic latency factors associated with manual passenger processing during flight disruptions?
- ii. How can an information retrieval system leveraging Retrieval-Augmented Generation (RAG) be mathematically and programmatically structured to eliminate conversational hallucinations regarding corporate policy?
- iii. How can database transactions leverage ACID properties to prevent data collisions when allocating flight seats under high-concurrency conditions?
- iv. How effectively can a semantic routing algorithm differentiate between autonomous capability limits and complex edge cases to manage a structured hand-off to human service teams?

1.4 Justifications of the Study

From an operational and organizational perspective, Kenya Airways stands to benefit substantially from the deployment of this architecture. By automating the resolution of typical, repetitive passenger queries during delays (such as flight status updates, digital voucher distribution, and route modifications), the airline can shift its response metrics from hours to a sub-five-minute window. This rapid intervention disperses terminal crowds, mitigates pressure on ground staff at JKIA, and allows human personnel to focus exclusively on highly complex customer edge cases.

From an economic perspective, the system minimizes financial leakages directly tied to flight delays. By accelerating passenger re-accommodation onto partner airlines or alternative routes, the system significantly curtails the "soft costs" of disruptions, such as extensive hotel accommodation overnights and long-term customer churn (BHI Research Group, 2025) . Additionally, by cross-referencing user profiles with the *Asante Rewards* database, the framework turns a negative operational event into a targeted revenue stream by intelligently proposing personalised, paid ancillary upgrades (e.g., premium lounge access, extra legroom) directly matching the consumer's tier preferences.

From an academic and computational perspective, this research expands the understanding of multi-agent orchestration frameworks within highly constrained, transactional enterprise environments. While most contemporary AI research focuses on generic, conversational models, this project establishes a concrete methodology for pairing nondeterministic language

model interfaces with entirely deterministic database transactional layers . It provides a blueprint that can modify mission-critical enterprise states without compromising data integrity.

1.5 Scope and Limitations

1.5.1 Scope of the Project

The functional boundaries of this project encompass the design and engineering of the core application brain via Python (FastAPI), a secure database layer via Supabase, a chat interface for web users, and a telephony interface via a dedicated voice API provider. The passenger workflow is bounded by four sequential technical stages which are

Secure Authentication to validate the user's secure identity strictly via their unique Passenger Name Record (PNR) code and surname matched against database records.

Policy-Grounded Concierge for managing real-time queries regarding terminal navigation, boarding schedules, and custom destination visa regulations using a RAG vector backend.

Atomic Disruption Recovery for automating seat selection and voucher generation triggered by API data alerts, managed via strict row-level database locks.

Intelligent Scope-Guarding for detecting out-of-scope input anomalies, terminating autonomous loops, and packaging data into a text-based "Case Brief" pushed to an internal agent dashboard alongside a synchronized telephonic voice routing link.

1.5.2 Limitations in the Project

The system performance and processing speed are very dependent on some external dependencies which are out of control. First, since the platform relies on the third-party foundational LLM APIs to perform the text vectorisation and semantic reasoning, the system responsiveness will be directly affected by any latency variations or infrastructure outages on the provider side. Second, the automated flight monitoring engine depends entirely on the accuracy of data from the external flight tracking APIs . Delays in global telemetry updates could delay the system's proactive outreach window. Finally, in the assessment of the telephony layer, the ability to transmit audio is still greatly affected by the congestion of the global telecom network, and by the strength of the roaming signals at the local terminals, which can lead to a variance in data translation within the automatic speech recognition (ASR) sub-modules.

1.5.3 Delimitations in the Project

This project explicitly excludes physical baggage routing, hardware RFID sorting tracking, and aircraft scheduling adjustments. The framework accepts operational reality as a given text input and will not alter aircraft route paths or crew scheduling matrices. Furthermore, while the system will calculate prices and offer paid premium ancillary tier upgrades, the architecture does not directly store, process, or route credit card payment variables; it handles all transactional logic up to the selection point, before offloading the final payment capture securely to Kenya Airways' existing certified external banking gateway.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the existing technology and literature related to how airlines manage passenger flight disruptions. It explores the transition from traditional, manual booking systems to modern Artificial Intelligence (AI) solutions. Section 2.2 examines the current challenges airlines face when flights are delayed or canceled, focusing on the high costs and customer frustrations caused by slow response times. Section 2.3 analyzes the technological tools currently used in the industry, such as standard telephone menus, basic automated chatbots, and traditional flight booking databases. Finally, this chapter will lay the groundwork for understanding why these older systems fail during mass disruptions and why a smart, fully automated AI system is necessary to solve the problem.

2.2 Current State of Disruption Recovery in Aviation

Managing Irregular Operations (IROPS) is an optimization challenge that requires balancing real-time resource availability against complex regulatory frameworks. When a flight disruption event occurs, scheduling systems must simultaneously reconcile crew tracking states, aircraft maintenance routing, and passenger consumer protections (BHI Research Group, 2025)

A major operational bottleneck stems from the manual parsing of airline policy documents during passenger re-accommodation. Document structures like the *Kenya Airways Conditions of Carriage* dictate distinct legal obligations based on variables like disruption source (e.g., internal maintenance vs. exogenous weather conditions) and passenger loyalty tiers.

Processing thousands of stranded travelers under these variable guidelines creates severe administrative delays, elevating customer dissatisfaction and driving up operational costs. This challenge demonstrates the need for digital automation frameworks that parse policy rules instantly while preserving core backend data integrity loyalty (1Point1 Solutions, 2025; Tech Mahindra, 2026).

2.3 Related Works in Airline Disruption Management

To understand why a new AI framework is needed, it is important to look at the automated tools airlines already use and understand where they fall short.

2.4 Traditional Telephone Menus (Legacy IVR Systems)

Traditional telephone menus, universally conceptualized as legacy Interactive Voice Response (IVR) systems, constitute the foundational automated telephonic portals that passengers confront when attempting to contact airline customer service infrastructures during operational

disruptions. These legacy frameworks prompt the user to navigate a pre-calculated, deterministic menu tree by executing dual-tone multi-frequency keypad selections or reciting monosyllabic, isolated phrases such as choosing a language preference or inquiring about an unadjusted flight status. From a technological perspective, these deployments operate over traditional public switched telephone networks and rely strictly on shallow keyword pattern matchers or rudimentary voice recognition software to advance the caller through a predefined programmatic loop (Auer et al., 2024).

The primary architectural gap inherent in these early telephonic frameworks is their absolute inability to parse or resolve non-linear, multi-variable customer crises within dynamic operational contexts. When an irregular operations crisis event occurs and a passenger requires sophisticated, cross-organizational re-accommodation—such as executing an immediate interline ticket re-booking onto a separate alliance partner carrier, extracting legal compliance allowances, or generating a localized emergency accommodation voucher—the system’s deterministic logic completely breaks down (Zhang et al., 2022). Because the rigid data pipeline lacks semantic parsing capabilities, context-aware memory state transitions, or dynamic backend transaction integration, it inevitably encounters an unhandled routing exception, terminating the automated loop and forcing the stranded traveler into prolonged hold queues to wait for manual human agent intervention (Gunarathne et al., 2018).

2.3.2 Basic Rule-Based Chatbots

Basic rule-based chatbots represent the initial class of simple, text-based automated interfaces deployed directly onto air carrier websites or instant messaging channels like WhatsApp, similar to the early conversational configurations utilized by regional airlines such as Kenya Airways. These entry-level systems are engineered almost exclusively to operate as digital document directories, answering high-frequency, standardized frequently asked questions (FAQs). From a technical perspective, these bots run on rigid string-matching architectures that scan a user's input for specific, hard-coded keywords and instantly return pre-written static scripts based on a strict programmatic flowchart.

The primary gap inherent in this rule-based approach is that because the systems are not truly intelligent, they fail completely whenever a passenger introduces a unique, nested, or complex question. Furthermore, attempting to upgrade this conversational layer by replacing it with standard, unconstrained generative artificial intelligence creates severe corporate vulnerabilities, as the underlying language model can hallucinate and confidently synthesize false airline policies—such as promising a stranded traveler an unearned free first-class

upgrade that they are not legally qualified to receive under the airline's conditions of carriage (Kellogg et al., 2025; Zheng et al., 2025).

2.3.3 Standard Flight Booking Databases (Legacy GDS)

Standard flight booking databases, professionally recognized as global distribution systems such as Amadeus or Sabre, constitute the massive, centralized global networks that human travel agents and airline operators rely upon to issue tickets, modify passenger records, and manage seat inventories. From a technological standpoint, these foundational infrastructures utilize traditional relational database structures and structured query languages to organize passenger name records alongside active aircraft capacities.

The primary architectural gap of these legacy databases is that while they remain highly secure and transactionally stable when operated manually by trained human service professionals, they create extreme systemic risks when integrated directly into an automated artificial intelligence multi-agent chat interface. If a high-speed conversational engine attempts to simultaneously re-book a volume of one hundred disrupted passengers onto a single rescue flight at the exact same millisecond, the lack of specialized, programmatic database locking rules will trigger severe data corruption, lost updates, and catastrophic seat booking errors (DB Designer, 2025).

Table 2.1: Summary of Existing Airline Systems and Their Flaws

System Type	What it does well	Where it fails during a crisis
Traditional Phone Menus	Very easy for anyone with a phone to access.	Cannot re-book tickets automatically; always requires a human.
Basic Chatbots	Good for answering simple, common questions.	Cannot understand complex problems; risks making up fake rules (hallucinations).
Traditional Databases	Securely stores all passenger data.	Cannot handle hundreds of automated AI requests at once without double-booking seats.

2.4 Gaps in Related Works

When evaluating legacy telephone menus, basic chatbots, and traditional database networks collectively, a distinct pattern of technological friction emerges. While these tools function adequately during normal, day-to-day airline operations, they consistently buckle under the sudden pressure of mass flight cancellations (Tech Mahindra, 2026).

The primary shortcoming of older automated systems lies in their inability to process complex, contextual reasoning (ALG Global, 2026). Traditional IVR menus and rule-based chatbots are bound by rigid programming loops. If a stranded passenger presents a unique scenario—such as missing a connecting flight that suddenly necessitates a transit visa—these systems cannot adapt. They ultimately default to transferring the user to a human agent, entirely defeating the purpose of automation.

Conversely, while modern Large Language Models (LLMs) possess the cognitive flexibility that older bots lack, they introduce a significant new risk into the aviation space: AI hallucinations (Lakera AI, 2026). If deployed without strict architectural boundaries, a generative AI model might confidently fabricate airline policies, inadvertently promising illegal compensation payouts (Moffatt v. Air Canada, 2024). Recent studies show that uncontrolled AI frequently suffers from "generation deficiency," creating rules that sound highly convincing but are factually incorrect (Zheng et al., 2025). Therefore, the industry

requires a system that physically restrains the AI, forcing it to generate answers derived exclusively from the airline's official rulebook (Kellogg et al., 2025).

Beyond conversational intelligence, there is a critical vulnerability at the database level. Exposing older database architectures to a high-speed AI agent during a major disruption invites catastrophic concurrency failures (Folgar, 2025). If hundreds of automated requests attempt to secure the few remaining seats on a rescue flight at the exact same millisecond, the lack of modern database locking mechanisms will result in lost updates and severe data collisions, effectively assigning the same physical seat to multiple passengers (DB Designer, 2025).

Ultimately, there remains a glaring unmet need for a unified framework. The aviation industry requires an architecture that seamlessly marries the conversational intelligence of modern AI, the strict policy adherence of Retrieval-Augmented Generation , and the secure, collision-free transaction controls of a modern database

2.5 Conceptual Framework

The conceptual framework for this study serves as the architectural blueprint for the Agentic AI Disruption Recovery system. It illustrates exactly how the proposed system plans to resolve the gaps identified in the current literature. By tracking the flow of data from the initial moment a passenger requests assistance to the final issuance of a new ticket, this framework highlights how the system balances conversational empathy with rigorous database security.

To successfully execute an automated passenger recovery workflow, the system orchestrates three distinct categories of variables, which work together as an integrated computational feedback loop. The initial stage of this loop is driven entirely by the input variables, which function as the core data sources necessary to contextualize the disruption. These variables include the multi-modal natural language query submitted by the stranded passenger via mobile text or telephonic voice audio streams, the passenger's unique alphanumeric Passenger Name Record (PNR) token to verify traveler identity and customer loyalty status, and the fully digitized, partitioned vector text of the Kenya Airways Conditions of Carriage document, which acts as the system's absolute corporate source of truth (Auer et al., 2024).

Once these inputs are successfully ingested, the processing variables take over as the central computational engine of the architecture to analyze, validate, and isolate data states. Within this processing layer, a Retrieval-Augmented Generation (RAG) engine transforms the passenger's raw question strings into high-dimensional mathematical vector coordinates to run high-speed similarity checks and locate the exact legal policy text block required for the

disruption. Simultaneously, an independent semantic guardrail variable constantly monitors the conversational distance metrics; if a traveler's query falls below a fixed similarity threshold, this safety engine stops the autonomous pipeline and immediately triggers a human hand-off routine over a live connection (Zheng et al., 2025). Concurrently, ACID-compliant database transaction parameters execute strict, pessimistic row-level locking routines at the persistence layer to freeze a target seat's database record the exact millisecond the automation selects it, guaranteeing that double-booking errors are mathematically impossible during high-concurrency reservation spikes (DB Designer, 2025).

The culmination of this multi-agent coordination results in the generation of explicit output variables, which represent the tangible operational recovery actions returned to the user interface. These output variables include the confirmed issuance and native digital rendering of a new, valid flight itinerary alongside the automated generation of cryptographic mobile meal or hotel service vouchers tailored precisely to the passenger's calculated delay duration and tier classification. Furthermore, in the event of an unexpected processing failure or a semantic policy violation, the system automatically outputs a structured Diagnostic Case Brief, which is forwarded instantly to human ground staff terminals over an open communication channel to ensure a continuous, seamless customer service transition.

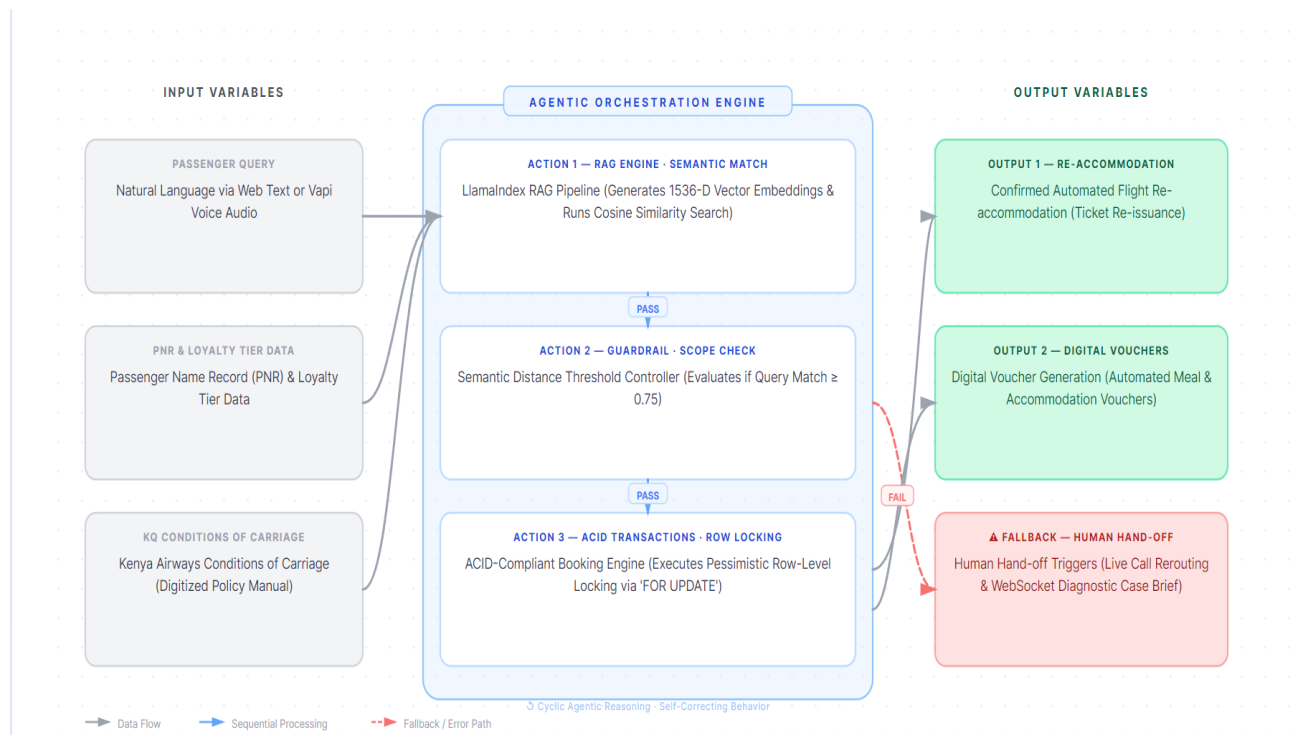


Figure. 2.1: Conceptual framework

Chapter 3: Development Methodology

3.1 Introduction

This chapter explains the step-by-step process, design choices, and testing methods used to build the autonomous AI framework. The main goal is to show how the system takes a passenger's request, processes it using intelligent software assistants, and safely updates the flight booking database without errors. The system balances natural, easy conversational communication with strict safety rules to handle flight changes automatically. The sections below explain the research approach, how data is gathered and cleaned, how the software assistants are organized, the project management lifecycle, and the specific database tools used to keep passenger data safe.

3.2 Research Paradigm

The AI framework uses a practical software engineering research approach that focuses on building and testing a real working solution to a major business problem. This approach provides a clear path to study how a mobile app can talk to a smart backend server to change flights safely without human help. Instead of just guessing how well the system works or letting the artificial intelligence make decisions completely on its own, this project adds strict safety limits. By combining flexible chat features with solid mathematical checks and strict database safety rules, the system creates a reliable setup for business use. The building process covers everything from receiving a passenger's message to testing how well the system works under heavy traffic.

3.2.1 Data Acquisition

The information used to build this project is divided into two main categories: official airline rules and simulated passenger files. The airline rules come from the official terms and conditions documentation used by commercial airlines, focusing specifically on rules for flight cancellations, luggage delays, food allowances, and hotel room setups. The passenger data is a simulated database built to look like a live airline booking system. This database includes fake customer profiles showing passenger identification codes, last names, exact flight paths, ticket types, and frequent flyer status levels such as blue, silver, gold, and elite tiers. A separate flight inventory table tracks individual seat numbers, airplane models, flight times, and airport codes.

3.2.2 Data Preprocessing

Before the system can read any text or files, the data must be cleaned and organized. The official airline rules document is stripped of messy formatting and cut into small, overlapping

text blocks that are easy for computers to read. These blocks are then turned into numbers that represent the meaning of the words and are stored in a specialized database so they can be looked up quickly later. On a separate path, when a user speaks a question into the mobile app, the system captures the voice audio file, evens out the sound quality, and uses a fast speech-to-text tool to turn the audio into clean text sentences that the backend server can process.

3.2.3 Model Training

The conversational part of the AI framework does not require training a massive artificial intelligence model from scratch. Instead, it uses smart software assistants that read the ready-made rules to answer questions. These software assistants are set up in a structured graph layout where each node represents a single job, like checking a passenger's identity or finding a matching flight. The assistants work in a strict loop and share a single digital notebook that tracks the conversation history, who the passenger is, and what rules apply to their situation, ensuring the system never makes random choices.

3.2.4 Model Validation and Testing

To stop the conversational system from making things up or promising wrong flight upgrades, the project uses strict mathematical safety checks. When the system pulls rule blocks from the database, it calculates how close the meaning of the user's question is to the actual airline policy using a similarity score.

The core computer science research contribution of this project lies in the mathematical validation of this semantic distance thresholding algorithm. When the system vectorizes a user query, it computes the Cosine Similarity against the embedded policy vectors (Reimers & Gurevych, 2019). Let $\mathbf{A}$ be the user query vector and $\mathbf{B}$ be the policy document vector. The similarity is calculated as:

$$= \text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

The system enforces a strict threshold limit. The research will experimentally evaluate the optimal threshold boundary to balance security against system utility. This is formalized using an F_β score optimization, which forces the system to prioritize minimizing false negatives (hallucinations) while keeping false positives (incorrect routing to human agents) reasonable (Karpukhin et al., 2020; Lewis et al., 2020):

$$F_\beta = (1 + \beta^2) \frac{\text{Precision} \times \text{Recall}}{(\beta^2 \times \text{Precision}) + \text{Recall}}$$

By setting $\beta = 0.5$, the methodology mathematically weights Precision higher than Recall, ensuring the architecture is algorithmically tuned to aggressively stop out-of-scope

queries. Any rule block that scores below the threshold is immediately rejected, stopping the response, and automatically handing the passenger over to a live human agent.

3.3 Software Development Methodology

The AI framework is built using Agile-Scrum, which is a popular project management method that breaks development work into short, two-week cycles called sprints. This method allows the team to update the software frequently and adapt to new requirements quickly. Each sprint begins with a meeting to pick jobs from a master list, guess how long they will take, and assign them to developers. The team holds brief daily update meetings to discuss any technical problems and keep the mobile app and server synchronized. At the end of every two-week sprint, the team reviews the working software and looks at performance data to plan the next phase of work.

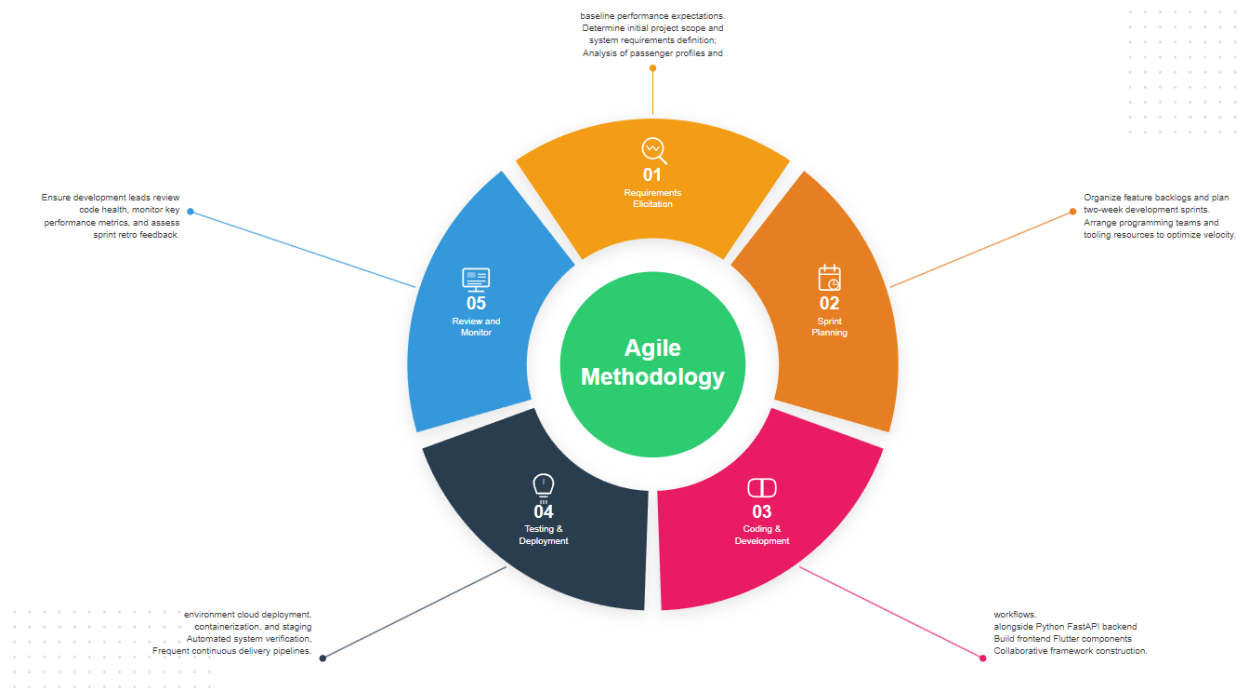


Figure. 3.1: Agile Methodology

3.3.1 Requirements Elicitation

The experimental baseline for the system was established by defining the strict computational limits required during high-traffic irregular operations (IROPS). The primary research requirements dictate that the architecture must process multimodal (text and voice) input while executing semantic vector searches against complex airline policy databases with sub-second latency. On the infrastructure side, the system mandates ACID-compliant transaction isolation,

ensuring zero data collisions during concurrent seat allocations, and network latency bounded below two seconds under maximum transactional load (Kleppmann, 2017).

3.3.2 Sprint Planning

Before technical execution begins, each two-week cycle is initialized with an experimental planning phase. Rather than standard feature selection, this phase defines the specific algorithmic parameters to be tested, such as adjusting the vector embedding dimensions or modifying the concurrency load thresholds. The tasks are decomposed into measurable computational metrics, ensuring the research objectives for the upcoming sprint are strictly aligned with evaluating the model's accuracy and database integrity.

3.3.4 Daily Standup and Tracking

To maintain experimental rigor and identify architectural bottlenecks early, brief daily synchronization protocols are executed. These checkpoints focus on tracking the stability of the backend orchestration, specifically monitoring database connection pooling, vector retrieval latency, and potential deadlocks within the state-machine logic. This continuous tracking ensures that the multimodal testing interfaces remain perfectly synchronized with the underlying mathematical models.

3.3.5 Sprint Review

At the conclusion of the sprint, the review phase functions as an empirical evaluation of the active algorithms. Instead of a standard software demonstration, the system undergoes rigorous testing of the semantic distance thresholding mechanism. Performance metrics, including the calculated F_{β} scores for the Retrieval-Augmented Generation (RAG) pipeline, are analyzed to ensure the system aggressively limits false negatives (hallucinations) while maintaining an acceptable rate of human hand-offs (Karpukhin et al., 2020; Lewis et al., 2020).

3.3.6 Sprint Retrospective

The final step of the cycle evaluates the computational efficiency of the development architecture. It is a period to analyze pipeline latencies, assess the algorithmic overhead of the cosine similarity calculations, and refine the database locking mechanics. By continuously optimizing the vector ingestion and querying methodologies, the underlying research model becomes more computationally efficient over the course of the project lifecycle.

3.3.7 Coding and Development

The construction phase strictly adheres to the established architectural framework. The mobile multimodal testing environment is engineered using Flutter and Dart, serving primarily to stream high-frequency concurrent API requests to the orchestration layer. The backend server

utilizes Python and FastAPI to construct high-throughput asynchronous endpoints. All source code governing the AI routing logic and deterministic rule parsers is managed via strict version control, requiring algorithmic unit tests to pass before integration into the primary research branch.

3.3.8 Testing and Deployment

The testing pipeline is designed to empirically validate the system's concurrency controls and retrieval accuracy under enterprise-level stress. Automated algorithmic load testing simulates high-concurrency race conditions, deploying tools like Apache JMeter to trigger hundreds of simultaneous Agentic AI requests. This validates the efficacy of the pessimistic FOR UPDATE database locks in preventing double-booking errors (Kleppmann, 2017). Upon successful validation, the research backend is containerized and deployed to an isolated cloud environment, bridging the experimental AI model with the multimodal testing client.

3.4 System Analysis and Design

The system analysis and design phase maps out the logical boundaries, data shapes, table links, and user paths that make the entire platform work.

3.4.1 Use Case Diagram

The system layout draws a clear line between two main types of users: the traveling passenger and the administrator desk operator. The passenger interacts with four main parts of the system: logging in with their booking code, typing or speaking questions, confirming a new flight, and downloading digital vouchers. The desk operator manages backend settings and steps in when there is a manual transfer. This manual transfer links directly to the automated flight change system, triggering automatically whenever the safety tools spot an error or an unresolvable flight issue.

3.4.2 Class Diagram

The structural design organizes system information into neat, well-defined code models to keep data organized. The system creates a passenger model that holds details like identification numbers, last names, and loyalty tier levels, which connects to a booking model. The booking model manages the booking reference code, the flight number, the assigned seat, and the status of the trip. This booking model links directly to a flight model, which tracks flight numbers, takeoff times, empty seats, and version numbers, as well as a voucher model that holds unique coupon codes, expiry dates, and cash values.

3.4.3 Entity Relationship Diagram (ERD)

The database structure is organized carefully to avoid messy data duplication and keep records clean. The data model separates records into distinct tables connected by strict mapping rules. The passenger profile table uses a unique ID number that links to a separate booking tracking table. The main booking table uses a combination of the booking code and flight number to link directly to a master flight schedule table. This setup ensures that if a flight time or passenger profile changes, the update spreads across all matching tables instantly without leaving broken or orphaned data entries behind.

3.4.4 Database Schema

The physical storage setup uses three main tables inside a standard SQL database. The passenger table contains columns for passenger ID numbers, last names, and loyalty levels. The flight table has columns for flight codes, departure timestamps, available seats, and a version counter to track record updates. The central booking table joins these rows together by tracking unique booking IDs, text reference codes, matching passenger IDs, flight codes, seat numbers, and update timestamps.

3.4.5 System Architecture

The technical setup splits the platform into three independent layers hosted in the cloud. The top layer is the client application, which displays the mobile user interface and manages user inputs. The middle layer is the backend service server, which runs the Python application, routes messages between software assistants, breaks down text words, and handles the speech-to-text tools. The bottom layer is the database platform, which stores standard tables right alongside the text meaning vectors. All information sent between these layers uses secure networks and encrypted messages, while fast chat updates use continuous, open network connections.

3.4.6 Wireframes

The mobile screen design focuses on a clean, simple layout to help reduce passenger stress during airport delays. The layout puts a status header at the top of the screen to display the passenger's name, loyalty level, and current flight alerts. The middle of the screen shows a scrollable chat area that separates the user's questions from the system's answers using colored text bubbles. The bottom of the screen contains a simple text entry box for typing alongside a large microphone button that lets users record and send voice messages easily.

3.5 System Development Tools and Techniques

The building phases of the project combines specialized software tools and modern development methods to create a fast, safe application.

3.5.1 Flutter

The mobile app is built entirely using the Flutter development kit and the Dart programming language. Choosing this setup allows the project to use a single set of source code files that can build native apps for both iOS and Android phones. Dart's advanced programming tools allow the app to handle multiple network requests at the same time without freezing the screen, ensuring a smooth visual experience for the user during fast text changes.

3.5.2 FastAPI

The high-performance server application is built using the FastAPI web framework and the Python programming language. FastAPI allows the server to handle thousands of incoming client requests simultaneously on a single thread by using non-blocking background loops. Integrated inside this Python server is a graph routing engine that manages the software assistants, allowing the system to break down tricky user requests into simple, step-by-step tasks.

3.5.3 Supabase

All standard information and text meaning profiles are stored inside a PostgreSQL database managed by the Supabase cloud platform. Supabase provides automated traffic managers to handle sudden, massive spikes in passenger usage. The addition of specialized vector tools inside the database allows the system to store text meaning models right inside standard data rows, letting the server run fast similarity searches alongside normal database updates.

3.5.4 Technique 1: Data-retrieval check

To stop the system from making up false statements and protect the airline from offering wrong services, the project uses a data-retrieval check based on mathematical comparison scores. When a user sends a message, the server turns the text into numbers and checks it against the official airline terms and conditions database. The system acts as a factual wall, rejecting any rule text that does not show a high matching score, ensuring the system only gives answers based on verified airline policies.

3.5.5 Technique 2: Database lock during transactions

To avoid database confusion, errors, and double-booking mistakes when thousands of delayed travelers try to claim the last remaining seats at the exact same moment, the system locks database rows during transactions. When a software assistant finds an available seat for a

passenger, it temporarily freezes that specific flight row in the database. This block stops all other automated assistants from writing to or changing that row until the passenger's ticket is successfully updated and their digital vouchers are saved safely to the disk.

3.5.6 Technique 3: Assistant Networks

The operational workflow uses smart assistant networks to break down complicated business logic into small, manageable steps. Instead of running code inside one long, rigid text script, the server passes data through a loop where each software assistant has a single job, like checking seat options or creating discount vouchers. Conditional rules check system values in real time, letting the network rerun steps automatically or hand the session over to a human agent instantly if a safety check fails.

3.6 System Deliverables

The development project produces five key software assets that combine to form the complete automated system.

3.6.1 System Documentation

A complete technical manual and code archive containing the full, commented source code files for both the Flutter mobile app and the Python backend server. This asset includes detailed installation guides, database setup rules, entity relationship diagrams, and official API specification maps.

3.6.2 Authentication Module

A separate security module that handles initial user sign-ins and session setup. The module reads booking reference codes and matching last names from the mobile screen, checks them against real records in the database, and creates a secure login token that holds the passenger's profile details to start the automated conversation safely.

3.6.3 Text and speech ingestion

The third primary asset is the speech and text ingestion module. This component handles network communication, taking text messages or raw voice recordings straight from the mobile app. It controls the phone's microphone hardware, evens out audio records, turns speech into text, and sends clean sentences to the server without slowing down the user interface.

3.6.4 Factual policy Engine

The fourth asset provides the safe policy lookup and factual response generation engine. This cloud backend module handles the database search pipeline and connects directly to the vector store. It tracks real-time similarity scores, runs the mathematical comparison filters, and stops the system from saying anything that is not directly found in the official airline rules document.

3.6.5 Secure transaction

The fifth asset provides the secure flight change execution module. This critical transaction engine handles database connection paths and manages row-level data locks. It controls the database update process, freezes flight seat rows during high-concurrency booking spikes, re-issues digital tickets, generates secure food and hotel coupons, and sends a clean confirmation message back to the passenger's mobile app screen instantly.

Reference

- ALG Global. (2026). [The AI trends that will shape aviation in 2026](#). *ALG Aviation Insights*.
- Auer, I., Schlögl, S., & Glowka, G. (2024). Chatbots in airport customer service—Exploring use cases and technology acceptance. *Future Internet*, 16(5), Article 175. <https://doi.org/10.3390/fi16050175>
- BHI Research Group (2025). [Disruption management in airline operations: A solver-based approach using time-space network optimization](#). *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 35(1), 412-420.
- Cook, A., Tanner, G., & Lawes, A. (2012). The European airline delay cost structure. *Journal of Air Transport Management*, 19, 7-13. <https://doi.org/10.1016/j.jairtraman.2011.12.005>
- Folgar, P. D. (2025). [The Concurrency Challenge: Managing High-Traffic Multi-User Systems](#). *Paradigma Digital Architecture Review*.
- Gunarathne, P., Rui, H., & Seidmann, A. (2018). When social media delivers customer service: Differential customer treatment in the airline industry. *MIS Quarterly*, 42(2), 489–420. <https://doi.org/10.25300/misq/2018/14290>
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769–6781. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
- Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media. <https://dataintensive.net/>
- Lakera AI. (2026). [LLM Hallucinations in 2026: How to Understand and Tackle AI's Most Persistent Quirk](#). *Lakera Enterprise Security Report*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Lewis, M., Riedel, S., & Kiela, D. (2020). [Retrieval-augmented generation for knowledge-intensive NLP tasks](#). *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459-9474.

Moffatt v. Air Canada, 2024 BCCRT 149 (CanLII). [Civil Resolution Tribunal Decision regarding Negligent Chatbot Misrepresentation](#).

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992. <https://doi.org/10.18653/v1/D19-1410>

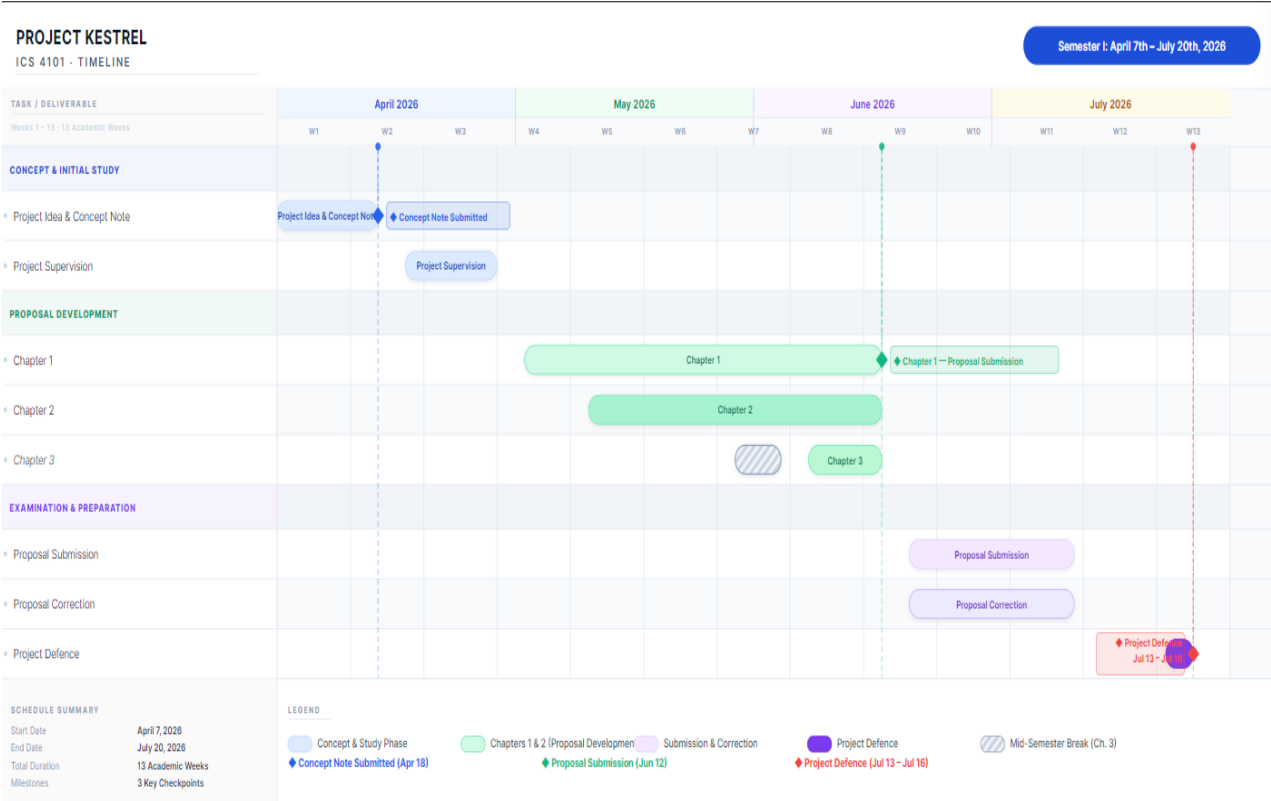
Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). [Database System Concepts](#) (7th ed.). McGraw-Hill Education.

Teodorović, D., & Guberinić, S. (1984). [Optimal dispatching strategy on an airline network after a schedule disruption](#). *European Journal of Operational Research*, 15(2), 178-182.

Zhang, Z., Li, B., & Liu, L. (2022). The impact of AI-based conversational agent on the firms' operational performance: Empirical evidence from a call center. *Applied Artificial Intelligence*, 37(1), Article 2157592. <https://doi.org/10.1080/08839514.2022.2157592>

Appendix

Appendix 1 : Gantt Chart



Appendix 1: Gantt chart

Appendix 2 : Similarity Report

Bill Ofieno

BRitchie_163362.docx

 Strathmore University (Main Account)

Document Details

Submission ID

trn:old::2945:392915473

Submission Date

Jun 12, 2026, 8:35 AM GMT+3

Download Date

Jun 12, 2026, 8:41 AM GMT+3

File Name

BRitchie_163362.docx

File Size

451.8 KB

34 Pages

7,595 Words

47,084 Characters

 Page 1 of 40 - Cover Page

Submission ID trn:old::2945:392915473

 Page 2 of 40 - Integrity Overview

Submission ID trn:old::2945:392915473

13% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

 **64 Not Cited or Quoted 12%**
Matches with neither in-text citation nor quotation marks

 **4 Missing Quotations 1%**
Matches that are still very similar to source material

 **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation

 **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

7%  Internet sources

3%  Publications

12%  Submitted works (Student Papers)

25